DSA0117 – DATA STRUCTURES AND ALGORITHMS USING C++

ASSIGNMENT – 2

CO2 – FUNCTIONS AND OBJECT-ORIENTED PROGRAMMING CONCEPTS IN C++

STUDENT DETAILS

Name: Madaraboina Gayathri

Register Number: 192472074

Course: Data Structures and Algorithms using C++

Assignment: Assignment 2 – CO2

Programming Language: C++

Compiler: g++ with C++17

IDE: Visual Studio Code

GITHUB Respository:https://github.com/GAYATHRI190506/DSA0117-C-/tree/main/DSA0117-192472074-Assignment2

1. INTRODUCTION

C++ is a powerful programming language that supports both procedural and object-oriented programming. Functions provide modularity and code reusability, while classes and objects help organize data and operations into a structured form.

This assignment focuses on function prototyping, call by value, call by reference, pointer parameters, return by reference, default arguments, function overloading, classes, member functions, access specifiers, arrays inside classes, static data members, static member functions, arrays of objects, friend functions, and inline functions.

The assignment also focuses on diagnosing an existing design, redesigning it using appropriate C++ concepts, and validating the resulting implementation through compilation and execution.

2. OBJECTIVES

1. Understand different methods of passing arguments to functions.
2. Differentiate call by value, call by reference, and pointer parameters.
3. Understand return by reference and its associated risks.
4. Implement default arguments and function overloading.
5. Implement classes, member functions, and arrays inside classes.
6. Understand static data members, static member functions, and arrays of objects.
7. Implement friend and inline functions.
8. Diagnose and redesign a C++ program using object-oriented concepts.
9. Compile, execute, and validate all programs.
10. Maintain the assignment using Git and GitHub.

3. SOFTWARE AND TOOLS USED

Tool	**Purpose**
C++	Programming language
g++	C++ compiler
C++17	Language standard
Visual Studio Code	Development environment
PowerShell	Compilation and execution
Git	Version control
GitHub	Repository and submission

4. ASSIGNMENT STRUCTURE

Task	**Topic**
Task 01	Call by Value, Reference and Pointer
Task 02	Return by Reference and Its Dangers
Task 03	Default Arguments and Function Overloading

Task 04	Class with Array Inside
Task 05	Static Members and Arrays of Objects
Task 06	Friend Functions and Inline Functions
Task 07	Diagnose, Redesign and Validate

5. TASK 01 – CALL BY VALUE, REFERENCE AND POINTER

Aim

To understand and demonstrate the difference between call by value, call by reference, and pointer parameters.

Program

```
#include <iostream>
using namespace std;

void byValue(int a) { a = 100; }
void byReference(int &a) { a = 100; }
void byPointer(int *a) { *a = 100; }

int main()
{
    int x = 1, y = 12, z = 13;
    byValue(x);
    byReference(y);
    byPointer(&z);
    cout << x << " " << y << " " << z << '\n';
    return 0;
}
```

Output

```
1 100 100
```

Explanation

Call by value changes only a copy. Call by reference modifies the original variable. A pointer parameter modifies the original variable through its address.

6. TASK 02 – RETURN BY REFERENCE AND ITS DANGERS

Aim

To understand return by reference and demonstrate the danger of returning a reference to a local variable.

Program

```
#include <iostream>
using namespace std;

int& safeReturn(int &x) { return x; }
```

```
int& dangerous()
{
    int local = 50;
    return local;
}

int main()
{
    int number = 100;
    cout << "Number = " << safeReturn(number) << '\n';
    return 0;
}
```

Output

```
Number = 100
```

Explanation

The dangerous() function returns a reference to a local variable. After the function ends, the local variable is destroyed, so the returned reference becomes invalid. The compiler warns about returning a reference to a local variable.

7. TASK 03 – DEFAULT ARGUMENTS AND FUNCTION OVERLOADING

Aim

To demonstrate default arguments and function overloading in C++.

Program

```
#include <iostream>
using namespace std;

int scholarship(int marks, int bonus = 0)
{
    return marks + bonus;
}

double scholarship(double marks)
{
    return marks;
}

int main()
{
    cout << "Scholarship with marks only: " << scholarship(80) << '\n';
    cout << "Scholarship with marks and bonus: " << scholarship(80, 10) << '\n';
    cout << "Scholarship with decimal marks: " << scholarship(85.5) << '\n';
    return 0;
}
```

Output

```
Scholarship with marks only: 80
Scholarship with marks and bonus: 90
Scholarship with decimal marks: 85.5
```

Explanation

The default bonus is 0 when no bonus is supplied. The double version is selected for decimal marks, demonstrating function overloading.

8. TASK 04 – CLASS WITH ARRAY INSIDE

Aim

To create a class containing an array and use member functions to calculate total and average marks.

Program

```
#include <iostream>
using namespace std;

class Student
{
private:
    int marks[5];

public:
    void inputMarks()
    {
        cout << "Enter 5 marks: ";
        for (int i = 0; i < 5; i++) cin >> marks[i];
    }

    int totalMarks()
    {
        int total = 0;
        for (int i = 0; i < 5; i++) total += marks[i];
        return total;
    }

    double averageMarks()
    {
        return totalMarks() / 5.0;
    }
};

int main()
{
    Student student;
    student.inputMarks();
    cout << "Total marks = " << student.totalMarks() << '\n';
    cout << "Average marks = " << student.averageMarks() << '\n';
    return 0;
}
```

Output

```
Enter 5 marks: 80 90 70 85 95
Total marks = 420
Average marks = 84
```

Explanation

The Student class stores five marks in a private array. Member functions calculate the total and average. For the given input, total = 420 and average = 84.

9. TASK 05 – STATIC MEMBERS AND ARRAYS OF OBJECTS

Aim

To demonstrate static data members, static member functions, and arrays of objects.

Program

```
#include <iostream>
using namespace std;

class Student
{
private:
    int rollNo;
    string name;
    static int count;

public:
    Student(int r, string n)
    {
        rollNo = r;
        name = n;
        count++;
    }

    void display()
    {
        cout << "Roll No: " << rollNo << ", Name: " << name << '\n';
    }

    static int getCount() { return count; }
};

int Student::count = 0;

int main()
{
    Student students[3] = {
        Student(101, "Gayathri"),
        Student(102, "Harini"),
        Student(103, "Rithika")
    };

    cout << "Number of students: " << Student::getCount() << '\n';
    cout << "\nStudent Details:\n";
```

```
    for (int i = 0; i < 3; i++) students[i].display();
    return 0;
}
```

Output

```
Number of students: 3

Student Details:
Roll No: 101, Name: Gayathri
Roll No: 102, Name: Harini
Roll No: 103, Name: Rithika
```

Explanation

The static count is shared by all objects and is incremented by the constructor. Three objects are stored in the students array, so the count is 3.

10. TASK 06 – FRIEND FUNCTIONS AND INLINE FUNCTIONS

Aim

To demonstrate friend functions and inline functions in C++.

Program

```
#include <iostream>
using namespace std;

class Student
{
private:
    int marks;

public:
    Student(int m) { marks = m; }
    friend void compareMarks(const Student& s1, const Student& s2);
};

void compareMarks(const Student& s1, const Student& s2)
{
    if (s1.marks > s2.marks)
        cout << "Student 1 has higher marks." << '\n';
    else if (s2.marks > s1.marks)
        cout << "Student 2 has higher marks." << '\n';
    else
        cout << "Both students have equal marks." << '\n';
}

inline int calculateBonus(int marks)
{
    return marks >= 90 ? 10 : 5;
}

int main()
{
```

```
    Student s1(85), s2(92);
    compareMarks(s1, s2);
    cout << "Bonus for Student 1: " << calculateBonus(85) << '\n';
    cout << "Bonus for Student 2: " << calculateBonus(92) << '\n';
    return 0;
}
```

Output

```
Student 2 has higher marks.
Bonus for Student 1: 5
Bonus for Student 2: 10
```

Explanation

The friend function can access the private marks members. The inline function returns 10 for marks of 90 or above and 5 otherwise.

11. TASK 07 – DIAGNOSE, REDESIGN AND VALIDATE

Aim

To implement a redesigned bank-account solution using constructors, member functions, static members, references, and a friend function.

Program

```
#include <iostream>
using namespace std;

class Account
{
private:
    int balance;
    static int accountCount;

public:
    Account(int b)
    {
        balance = b;
        accountCount++;
    }

    void deposit(int amount) { balance += amount; }
    int getBalance() const { return balance; }

    static int getAccountCount() { return accountCount; }

    friend void transfer(Account& from, Account& to, int amount);
};

int Account::accountCount = 0;

void transfer(Account& from, Account& to, int amount)
{
    if (amount > 0 && amount <= from.balance)
```

```
    {
        from.balance -= amount;
        to.balance += amount;
    }
}

int main()
{
    Account a1(5000);
    Account a2(3000);

    cout << "Initial Balance of Account 1: " << a1.getBalance() << '\n';
    cout << "Initial Balance of Account 2: " << a2.getBalance() << '\n';

    a1.deposit(1000);

    cout << "After Deposit - Account 1: " << a1.getBalance() << '\n';

    transfer(a1, a2, 2000);

    cout << "After Transfer - Account 1: " << a1.getBalance() << '\n';
    cout << "After Transfer - Account 2: " << a2.getBalance() << '\n';
    cout << "Total Accounts: " << Account::getAccountCount() << '\n';

    return 0;
}
```

Output

```
Initial Balance of Account 1: 5000
Initial Balance of Account 2: 3000
After Deposit - Account 1: 6000
After Transfer - Account 1: 4000
After Transfer - Account 2: 5000
Total Accounts: 2
```

Explanation

The program uses encapsulation, a constructor, a static data member, a static member function, a friend function, and reference parameters. The transfer condition prevents invalid transfers. The actual execution matched the expected output.

12. COMPARISON OF FUNCTION PARAMETER METHODS

Method	What is passed?	Original value changed?
Call by Value	Copy of value	No
Call by Reference	Reference to variable	Yes
Pointer	Address of variable	Yes
Return by Reference	Reference to existing value/object	Can modify original

13. IMPORTANT C++ CONCEPTS LEARNED

Function Prototype
Tells the compiler about a function before it is used.

Default Argument
Provides a value automatically when an argument is not supplied.

Function Overloading
Allows functions with the same name but different parameter lists.

Class and Object
A class is a user-defined type; an object is an instance of a class.

Access Specifiers
private, public, and protected control access to class members.

Static Member
A class-level member shared among objects.

Friend Function
A non-member function allowed to access private class members.

Inline Function
A small function for which the compiler may substitute the function body at the call site.

14. VALIDATION AND TESTING
All seven tasks were compiled and executed using g++ with C++17. Expected outputs were compared with actual outputs. Task 02 also demonstrated the compiler warning associated with returning a reference to a local variable.

```
g++ -std=c++17 -Wall -Wextra filename.cpp -o filename
.\filename.exe
```

15. TIME AND SPACE COMPLEXITY

Task	Time	Space
Task 01	O(1)	O(1)
Task 02	O(1)	O(1)
Task 03	O(1)	O(1)
Task 04	O(n)	O(n)
Task 05	O(n)	O(n)
Task 06	O(1)	O(1)
Task 07	O(1)	O(1)

16. GIT VERSION CONTROL

Git was used to maintain the assignment progressively. Each completed task was added and committed. The final Task 07 commit was 7ccee25.

```
git add Task07
git commit -m "Complete Task 07 - Diagnose Redesign and Validate"
```

The README file was also committed. The README commit was dd48ca7. The final local working tree was clean.

17. REPOSITORY STRUCTURE

```
Assignment2
├── .gitignore
├── README.md
├── Task01
│   ├── task01.cpp
│   └── output.png
├── Task02
│   ├── task02.cpp
│   └── output.png
├── Task03
│   ├── task03.cpp
│   └── output.png
├── Task04
│   ├── task04.cpp
│   └── output.png
├── Task05
│   ├── task05.cpp
│   └── output.png
├── Task06
│   ├── task06.cpp
│   └── output.png
└── Task07
    ├── task07.cpp
    └── output.png
```

The .gitignore file contains:

```
*.exe
*.o
desktop.ini
.vscode/
```

18. LEARNING OUTCOMES

11. Understood values, references, and pointers.
12. Understood return-by-reference and dangling references.
13. Implemented default arguments and function overloading.

14. Created classes and objects with access control.
15. Implemented arrays inside classes and arrays of objects.
16. Used static data members and static member functions.
17. Implemented friend and inline functions.
18. Applied encapsulation and reference parameters.
19. Validated programs through compilation and execution.
20. Used Git for version control and progressive commits.

19. CONCLUSION

This assignment provided practical experience with important C++ function and object-oriented programming concepts. The seven tasks demonstrated call by value, call by reference, pointer parameters, return by reference, default arguments, function overloading, classes, arrays inside classes, static members, arrays of objects, friend functions, and inline functions.

The final task combined multiple concepts into an Account management program. Encapsulation was achieved by keeping account balances private, a static member maintained the number of accounts, and a friend function performed transfers between accounts.

All programs were compiled using C++17 and executed successfully. The actual outputs were compared with the expected outputs, and the programs produced the intended results.

Therefore, the assignment successfully demonstrated the practical application of the required CO2 concepts in C++.

20. FINAL RESULT

Item	**Status**
Assignment	Completed Successfully
Number of Tasks	7
Compilation	Successful
Execution	Successful
Output Validation	Successful
Git Version Control	Implemented
README	Completed
Git Working Tree	Clean